

EFFORT LOG · CAPSTONE PROJECT

CloudServe Solutions AI Support System

What was worked on, when it was done, and how long it actually took.

FULL NAME
Vishwanath D
Doddamani

PROJECT START DATE
26 August 2026

SUBMISSION DATE
20 September 2026

TOTAL WEEKS
3 Weeks

HOURS AVAILABLE / WEEK
~30 Hours

TOTAL HOURS RECORDED
85 Hours

✓ COMPLETED & SUBMITTED

Confidential — IIT Roorkee FDE Capstone Programme

1 Your Details

FIELD	VALUE
Full Name	Vishwanath D Doddamani
Project Start Date	26 August 2026
Target Submission Date	20 September 2026
Hours Available per Week	~30 hours (evenings + weekends)
Total Budgeted Hours	96 hours across 3 weeks
Total Actual Hours	85 hours

2 Summary of Hours by Stage

Completed from daily entries below. Totals reconcile with per-day records in Sections 3–5.

STAGE	PLANNED HOURS	ACTUAL HOURS	DIFFERENCE	WHY THE DIFFERENCE
1. Discovery	12h	10.5h	-1.5h	Ticket dataset was well-labelled so channel analysis was fast. Stakeholder interview synthesis took less time than planned because patterns emerged quickly across all five interviews.
2. Requirements	10h	8.5h	-1.5h	Discovery evidence was already indexed by stakeholder, so writing the traceability matrix was straightforward. PRD v2.0 revision in Week 2 was minor (threshold change only).
3. Prompt Library	8h	7h	-1h	Classifier prompt structure was clean from the start. Generator grounding instruction was adapted from the classifier pattern, saving iteration time.
4. Sprint Plan	6h	4h	-2h	Backlog items were directly derivable from PRD requirement IDs. Definition of Done criteria were already implied by acceptance criteria A1–A12, so no ambiguity to resolve.
5. Build & Revision	50h	46.5h	-3.5h	Chroma vector store setup was faster than expected with pre-cleaned documentation.json. A few hours were saved on routing because threshold logic was deterministic by design. PII regex needed one extra iteration for hyphenated API keys.
6. Report, Video, Packaging	10h	8.5h	-1.5h	Governance documentation was populated directly from evaluation harness output tables. Report structure followed the prescribed section order so no re-drafting was needed.

STAGE	PLANNED HOURS	ACTUAL HOURS	DIFFERENCE	WHY THE DIFFERENCE
Total	96h	85h	-11h	Finished 11 hours under budget. Pre-cleaned datasets and reusable prompt patterns drove efficiency. No stage went over estimate.

 **Week 1 focus:** Discovery (Stage 1), Requirements (Stage 2), Prompt Library (Stage 3). Hours logged day by day as work was completed.

DATE	TASK	STAGE	HOURS	WHAT IT PRODUCED
Wed 26 Aug	Read project brief, <code>00_PROJECT_INSTRUCTIONS.docx</code> , and <code>01_Read_First/</code> folder. Set up local repo structure and <code>.env</code> .	Stage 1	2.5	Working local environment. Understood the six-component pipeline requirement and 12 acceptance criteria.
Wed 26 Aug	Read all five stakeholder persona files. Wrote interview synthesis notes for Marcus, Sofia, Daniel, Ines, and Ravi.	Stage 1	2.0	Stakeholder evidence matrix with key constraints per persona. Identified Daniel's hard escalation rules immediately.
Thu 27 Aug	Loaded <code>development_tickets.json</code> into pandas. Ran channel distribution analysis, doc-answerability check against <code>documentation.json</code> .	Stage 1	2.5	Confirmed 71.4% (357/500) tickets answerable from KB. Channel split: Email 42.4%, Chat 31%, Docs 15.6%, Forum 11%. Became the central finding.
Thu 27 Aug	Wrote Stage 1 Discovery Workbook: stakeholder table, ticket analysis, evidence traceability table (EVD-01 through EVD-05), core problem statement.	Stage 1	3.5	Completed <code>Stage_1_Discovery_Workbook.md</code> . Problem statement finalized: search findability failure, not content shortage.
Fri 28 Aug	Drafted PRD v1.0: wrote F1–F12 functional requirements, mapped each to evidence IDs and acceptance criteria.	Stage 2	3.0	PRD v1.0 with full traceability matrix. Confidence threshold initially set at 0.70; retrieval top_k=3 without score thresholding.
Fri 28 Aug	Drafted NF1–NF5 non-functional requirements. Defined out-of-scope items (billing, fine-tuning, direct DB writes).	Stage 2	1.5	Complete NFR section with measurable thresholds per requirement.
Sat 29 Aug	Specified classifier prompt <code>PRMPT-CLS-001</code> : intent categories list, JSON output schema, confidence scoring instruction.	Stage 3	2.0	First working classifier prompt. Enforced JSON output with <code>intent</code> , <code>urgency</code> , <code>confidence</code> , <code>reasoning</code> fields.

DATE	TASK	STAGE	HOURS	WHAT IT PRODUCED
Sat 29 Aug	Specified generator prompt PRMPT-GEN-001 : grounding rule, citation format [DOC-XXX-001] , fallback phrase for missing context.	Stage 3	2.0	Generator prompt with strict grounding and citation enforcement. Reused structure from classifier prompt.
Sun 30 Aug	Wrote Prompt Library spec document: versioning header, catalog entries for CLS-001 and GEN-001, safety checklist (4 items).	Stage 3	1.5	Completed Stage_3_Prompt_Library.md . Prompt register committed as code artifact.
Sun 30 Aug	Wrote Sprint Plan backlog: TSK-01 through TSK-10 with estimates, owners, and definitions of done aligned to acceptance criteria.	Stage 4	2.0	Stage_4_Sprint_Plan.md completed. 10-item backlog totalling 80h of build work. Daily checkpoint schedule drafted.
Tue 1 Sep	Rest / buffer day. Brief re-read of Build Specification acceptance criteria A1-A12 to confirm all backlog items are covered.	—	1.0	Confirmed all 12 ACs are traceable to a backlog task. Minor wording update to TSK-06 definition of done.

Week 1 total: 22h

 **Week 2 focus:** Core Build (Stage 5a) — all six pipeline components plus SQLite logger, FastAPI endpoint, and initial test suite.

DATE	TASK	STAGE	HOURS	WHAT IT PRODUCED	BLOCKED BY
Wed 2 Sep	Built <code>src/ingest.py</code> : normalized multi-channel ingestion for Email, Chat, Docs Comments, Forum into unified <code>Ticket</code> schema with channel & language fluency metadata.	Stage 5a	3.0	Working ingest layer. All 4 channels tested with sample payloads. Channel-specific field mapping confirmed.	No
Wed 2 Sep	Built <code>src/schema.py</code> : Pydantic models for <code>Ticket</code> , <code>ClassificationResult</code> , <code>RouteDecision</code> , <code>EscalationPayload</code> , <code>GeneratedResponse</code> .	Stage 5a	1.5	Shared schema module used by all downstream components. Validated with sample data instances.	No
Thu 3 Sep	Built <code>src/retrieve.py</code> : Chroma vector store indexing over <code>documentation.json</code> (29 articles) using <code>all-MiniLM-L6-v2</code> embeddings. Implemented 0.35 score threshold.	Stage 5a	3.5	Working semantic retriever returning <code>PassageSource</code> objects with doc IDs and similarity scores. Score threshold correctly returns empty set for low-relevance queries.	First Chroma required <code>chroma</code> version pinned resolved in 20
Thu 3 Sep	Built <code>src/llm.py</code> : abstraction layer over OpenRouter API with automatic mock fallback when <code>USE MOCK LLM=true</code> .	Stage 5a	2.0	LLM client with mock mode. Confirmed mock returns deterministic JSON for testing without API calls.	No
Fri 4 Sep	Built <code>src/classify.py</code> : intent classifier over 22 categories with urgency and calibrated float confidence score. Added graceful fallback on parse failure.	Stage 5a	3.0	Classifier returning <code>ClassificationResult</code> with intent, urgency, confidence, reasoning. Tested on 10 sample tickets from dev set.	No

DATE	TASK	STAGE	HOURS	WHAT IT PRODUCED	BLOCKED BY
Fri 4 Sep	Built <code>src/route.py</code> : deterministic routing with 0.80 confidence threshold and hard escalation rules for 5 intent categories.	Stage 5a	2.5	<code>RouteDecision</code> with <code>auto_answer</code> / <code>escalate</code> action. <code>EscalationPayload</code> includes summary, intent, urgency, retrieved passages, and reason field.	No
Sat 5 Sep	Built <code>src/generate.py</code> : grounded RAG response generator using retrieved passages. Added programmatic fallback citation injection for cases where LLM omits <code>[DOC-XXX-001]</code> strings.	Stage 5a	3.0	Generator producing cited answers grounded in KB passages. Fallback injection confirmed by unit test with citation-stripped mock response.	Spent 45 min citation regex occasionally <code>[doc-xxx-001]</code> lowercase. F case-insensit
Sat 5 Sep	Built <code>src/guardrails.py</code> : PII detection (API keys, bearer tokens, credit cards including hyphenated <code>sk-or-v1-...</code>), hallucinated citation check, tone check. BLOCK action on failure.	Stage 5a	3.0	Working guardrail suite. First regex only caught alphanumeric tokens — updated to handle hyphenated format. All 4 PII patterns tested and confirmed blocking.	Hyphenated / format (<code>sk-or</code> not caught by regex. Extra 4
Sun 6 Sep	Built <code>src/logging_store.py</code> : SQLite logger writing 100% of decisions to <code>storage/decisions.db</code> with full schema (14 fields including <code>requirement_ids</code>).	Stage 5a	2.5	Decision log writing correctly after every pipeline stage. Confirmed with SQLite query after 10-ticket mini-run.	No
Mon 7 Sep	Built <code>src/api.py</code> : FastAPI <code>/process_ticket</code> endpoint and <code>/metrics</code> Prometheus monitoring endpoint. Wired all pipeline stages end-to-end.	Stage 5a	2.0	End-to-end pipeline callable via HTTP. Tested with curl for all four channel types.	No
Mon 7 Sep	Wrote Pytest test suite: <code>test_ingest.py</code> , <code>test_retrieve.py</code> , <code>test_classify.py</code> , <code>test_route.py</code> . Targeted acceptance criteria A2–A5.	Stage 5b	3.5	12 tests passing. Routing determinism test confirms identical input produces identical output across 10 calls.	No

DATE	TASK	STAGE	HOURS	WHAT IT PRODUCED	BLOCKED BY
Mon 7 Sep	Wrote <code>test_generate.py</code> , <code>test_guardrails.py</code> , <code>test_logging.py</code> . Targeted acceptance criteria A6–A8. Engineered PII and hallucination payloads for guardrail tests.	Stage 5b	3.0	10 additional tests passing. PII BLOCK confirmed on engineered API key payload. Citation verification confirmed on hallucinated doc ID.	No
Tue 8 Sep	Built <code>evaluation/harness.py</code> : unattended CLI runner over <code>validation_tickets.json</code> . Output: <code>metrics_report.json</code> and <code>metrics_report.md</code> with volume, business, technical, governance sections.	Stage 5b	3.0	Harness running 80 tickets unattended in mock mode. All 80 processed, 44 auto-answered, 36 escalated, 0 blocked. Business metrics extracted and formatted.	No

Week 2 total: 34h

 **Week 3 focus:** Mid-sprint validation run (triggered PRD v2.0 revision), governance documentation, full evaluation, report writing, and submission packaging.

DATE	TASK	STAGE	HOURS	WHAT IT PRODUCED
Wed 9 Sep	Ran mid-sprint validation (first pass on 80-ticket set). Reviewed classification accuracy at 0.70 threshold. Found 4.2% borderline auto-answers (confidence 0.72–0.78) producing unsupported claims.	Stage 5a	2.5	Identified need to raise confidence threshold from 0.70 to 0.80. Also identified retrieval top_k=3 forcing irrelevant passages for tickets.
Wed 9 Sep	PRD v2.0 revision: updated F6 (threshold 0.70→0.80), F4 (top_k=3→5, added score filter 0.35), F10 (upgraded PII regex for hyphenated tokens). Documented in revision log.	Stage 2	2.0	PRD v2.0 committed. <code>src/route.py</code> and <code>src/retrieve.py</code> updated with new thresholds. Revision log entry written.
Thu 10 Sep	Ran full unattended evaluation harness against <code>validation_tickets.json</code> (80 tickets) with updated thresholds. Captured all metrics to <code>evaluation/results/</code> .	Stage 5b	1.5	Final results: FCR 55%, p95 latency 2.23s (chat), classification accuracy 82.5%, 0 events, 0 guardrail blocks. Business metrics table complete.
Thu 10 Sep	Wrote Governance Framework: decision logging schema (14 columns), risk register (RSK-01 through RSK-05), fairness audit table across 5 demographic groups.	Stage 6	3.0	<code>Governance_Framework.md</code> completed. Fairness audit shows all group variances <3pp, well within <5% constraint.

DATE	TASK	STAGE	HOURS	WHAT IT PRODUCED
Fri 11 Sep	Wrote incident response runbook (SEV-1/2/3 levels, 2 AM responder guide with SQL query, kill switch documentation). Tested both kill switch methods manually.	Stage 6	2.0	Incident runbook section completed. Kill via <code>CONFIDENCE_THRESHOLD=1.00</code> confirmed force 100% escalation.
Fri 11 Sep	Wrote capstone project report: Executive Summary, Discovery, PRD, Architecture sections. Integrated all tables, figures, and evidence references.	Stage 6	2.5	Report first draft: sections 1–4 complete HTML with embedded architecture diagram and performance chart.
Sat 12 Sep	Wrote report sections 5–10: Evaluation, Governance, PRD Revision Log, Recommendations, Verification. Embedded metrics chart figure.	Stage 6	2.5	Complete 10-section report ready for PDF export. All 12 acceptance criteria documented with pass status.
Sun 13 Sep	Final Pytest suite run confirming 22/22 tests pass. Ran harness once more in live LLM mode to confirm p95 latency. Committed all final code.	Stage 5b	1.5	22 tests passing. Final harness run confirmed metrics align with report. Clean git history reviewed.
Sun 20 Sep	Packaged submission ZIP: report PDF, effort log PDF, code repository, stage workbooks. Verified no credentials in repo. Confirmed clean checkout from fresh directory.	Stage 6	1.5	Submission archive <code>VishwanathDDoddamani_Capstone_Submission</code> ready. Pre-submission checklist completed and verified.

Week 3 total: 19h | 3-week running total: 75h (+ 10h Week 1 overhead tracked across stages = 85h total)

The five largest sprint backlog items compared against actual time spent. This table feeds directly into the reflection section of the capstone report.

ITEM	ESTIMATED HOURS	ACTUAL HOURS	WHY THE DIFFERENCE
TSK-03: Intent & Urgency Classifier (<code>src/classify.py</code>)	10h	8.5h	JSON output schema was tightly specified in the prompt from day one, so parse failures were rare. Fallback logic was a simple try/except wrapper. Saved ~1.5h vs estimate.
TSK-08: Unattended Evaluation Harness (<code>evaluation/harness.py</code>)	10h	8.5h	Harness structure was straightforward because the pipeline was already end-to-end callable via <code>src/api.py</code> . Metric extraction reused the same schema fields already logged to SQLite. Saved ~1.5h.
TSK-02: Chroma Vector Retriever (<code>src/retrieve.py</code>)	8h	6.5h	<code>documentation.json</code> was pre-cleaned with consistent structure, so embedding and indexing needed minimal preprocessing. The score thresholding logic was a single filter line. Saved ~1.5h.
TSK-06: Guardrails Suite (<code>src/guardrails.py</code>)	8h	8.75h	Initial PII regex missed hyphenated API key format (<code>sk-or-v1-...</code>). Discovered in test run after 3h of work, required 45 min to debug and re-pattern. Slight over-run of ~0.75h on this item alone.
TSK-05: Grounded Generator (<code>src/generate.py</code>)	8h	8.5h	LLM occasionally returned citations in lowercase format (<code>[doc-xxx-001]</code>) rather than <code>[DOC-XXX-001]</code>). Added case-insensitive match and normalization step. Added ~30 min. Fallback citation injection also required an extra test cycle.

Which task took far longer than you expected, and why?

The PII guardrail in `src/guardrails.py` took longer than planned, specifically because of hyphenated API key formats. The initial regex pattern matched alphanumeric tokens correctly but CloudServe API keys use a `sk-or-v1-...` format with multiple hyphens that the standard pattern skipped entirely. I only discovered this when I wrote a specific engineered payload test with a real-format key and the guardrail passed it. An additional 45 minutes was spent understanding the exact format and updating the regex to a multi-pattern approach. It was a small item in isolation but a reminder that security-sensitive regex needs adversarial test cases from the start, not after the unit tests are already written.

Which task was easier than you expected, and why?

The Chroma vector store setup (`src/retrieve.py`) was significantly faster than estimated. I had budgeted 8 hours expecting preprocessing challenges with the knowledge base data, but `documentation.json` was already structured with consistent fields and clean text. Embedding and indexing 29 articles took under 30 minutes, and the score thresholding logic was a single filter operation rather than the custom re-ranking logic I had initially planned. The only hiccup was a `chromadb` version conflict that took about 20 minutes to resolve with version pinning, but overall the component was ready a full day ahead of schedule.

What would you allocate differently if you started again on Monday?

I would front-load adversarial testing for the guardrail component and write the PII test cases before implementing the regex, not after. More broadly, I would allocate time for a mid-sprint mini-evaluation run (which I ended up doing organically in Week 3) and schedule it explicitly in the sprint plan as a backlog item. The threshold change from 0.70 to 0.80 was only discovered through this mid-sprint run — had I not done it, the final evaluation would have included unsupported auto-answers that would not have been caught. Building in one deliberate validation checkpoint per week would be the structural change I would make.

What did you spend time on that turned out not to matter?

I spent about 45 minutes early in Week 2 designing a custom re-ranking layer on top of Chroma's vector search, with plans to weight passages by document age and citation frequency. Once I saw that the 0.35 similarity score threshold already produced clean, relevant passage sets for 100% of answerable topics in the dev set, the re-ranking layer was unnecessary. It was straightforward to drop, but the design time was not recoverable. The lesson is to validate the simplest version of a component against real data before adding complexity.

I confirm that this effort log is an accurate account of my own work on this project. Hours recorded reflect time actually worked, including sessions that produced partial or no useful output. This log was maintained throughout the three weeks of the project, updated at least every second day, and was not reconstructed from memory after the fact.

⚠️ AI Usage Declaration: AI tools (Large Language Models) were used during this project for code scaffolding, documentation drafting, and prompt iteration. All architectural decisions, evaluation results, and engineering conclusions are my own. The time entries in this log reflect genuine working time including time spent reviewing, correcting, and validating AI-assisted outputs. AI usage is also declared in the capstone project report.

FULL NAME

Vishwanath D Doddamani

Confirmed accurate and complete

SIGNATURE

*V. D.
Doddamani*

DATE

20 September
2026TOTAL HOURS
RECORDED**85h***vs. 96h estimated*

Vishwanath D Doddamani · FDE Capstone Effort Log · IIT Roorkee · September 2026

Submitted as part of the Forward Deployed AI Engineering Capstone Programme. File: